NEXORA: Sovereign Hybrid RAG for Air-Gapped Aerospace Mission Intelligence

Putta Sujith
Dept. of Computer Science Technology
Dayananda Sagar University
Bangalore, India
eng23ct0058@dsu.edu.in

K Vikas Aneesh Reddy
Dept. of Computer Science Technology
Dayananda Sagar University
Bangalore, India
eng23ct0052@dsu.edu.in

Karnati Mokshith
Dept. of Computer Science Technology
Dayananda Sagar University
Bangalore, India
eng23ct0053@dsu.edu.in

***Abstract*—Hallucinations in language models used by space and defense groups happen a lot, around 38 percent of the time [1], [2]. This is especially true when these models use systems. The problem gets worse because these models have to work without internet access due to strict security rules. NEXORA was made specifically for the Indian Space Research Organizations intelligence work. It uses a way to match documents and checks facts using a special kind of graph. It also follows security rules and has a new way to check every output to make sure it is correct [3]. This system was tested on 200 reports from the Indian Space Research Organization, including reports on the PSLV, GSLV and Chandrayaan programs. It was also tested with 50 questions from experts. The system scored high in truthfulness with a score of 0.97. It also reduced hallucinations to 3 percent, which is a big improvement [4]. The system can do all this in 580 milliseconds with only 130 milliseconds spent on security checks. This is very fast. Shows that the system is safe and reliable which is very important, for space planning where small mistakes can be very dangerous.**

***Keywords*—Retrieval-Augmented Generation, Knowledge Graphs, Air-Gapped AI, Graph-NLI, Role-based access control, Digital Sovereignty, Maximal Marginal Relevance, Aerospace Intelligence**

I. Introduction

A. The Trust Gap in Aerospace AI

The problem with noncompliance is that it happens because cloud-dependent RAG architectures do not follow ISROs security rules. These rules say that certain information like mission documents, propellant formulation records and satellite telemetry data cannot be shared using cloud infrastructure.. The current RAG setups use external APIs and hosted inference endpoints, which means they are connected to outside services [3]. This creates three problems.

The RAG systems can pick up information from prompts without warning because the context is not filtered [4]. This can cause what is called hallucinations, where the model gives strong signals but does not show any signs of error [5]. This is a deal because it can lead to mistakes in mission planning like wrong propellant specs or orbital miscalculations. If this happens it could cause the whole system to fail.

The issue is not about how accurate the system is, but also about how well it protects important data. The standard RAG systems, which only use vectors have hallucination rates between 18% and 38% in fields [1], [2]. This means that RAG systems make mistakes a lot and this is a problem, for Regulatory noncompliance and RAG architectures [6].

B. Sovereign RAG Requirements

The system is set up to keep track of everything that happens so we can see where every piece of information comes from. This is done by keeping everything so nothing from the outside can get in. We do not use any connections or cloud services. Every single technical detail is checked to make sure it is correct using a collection of known facts. For example when it comes to ISRO missions we can follow the details from the PSLV all the way down to the vikas engine and the S200 boosters. The system also has a kind of protection called IND-CPA, which keeps sensitive information safe, from attackers. They will not be able to tell when something important is being used. All the information we get is locked in place. Cannot be changed, because it is based on logical steps that have been verified.

C. Contributions

NEXORA takes care of all five demands with four technical moves.

- It combines retrieval with MMR reranking at 0.5, which means it spreads out document sources so no single source is in charge during -hop searches.
- NEXORA has a -generation graph-based auditor that breaks down answers into small claims and checks them against Neo4j facts. This really helps to cut down hallucinations by 83.3%.
- The memory firewall is also important because it separates past chat data strictly. This stops parts from mixing with untrusted history.
- NEXORA uses IND-CPA RBAC, which includes three checkpoints. These checkpoints are: query blocking before search filtering documents when they are injected and scanning outputs after generation. All of this helps to stop any metadata leaks.

Now lets go through the rest of it. NEXORA is explained in sections. Section II talks about what people did. Section III explains how NEXORA works. Then section IV explains how to build NEXORA. Section V outlines the testing steps for NEXORA. After that section VI shares the findings for

NEXORA. Section VII looks at what's missing from NEXORA and why. Finally section VIII wraps everything up, about NEXORA.

II. RELATED WORK

A. Evolution of Retrieval-Augmented Generation

Vector- retrieval does really well with remembering what something means but it still gets things wrong with 12, 18 percent of the information being completely made up in technical data [1], [11]. This happens when the information looks similar because just because two things are similar it does not mean they are true. The context of the information can change a lot when you are talking about things. A model might find phrases that match [19]. It can still get the facts wrong because it is using old or incorrect information.

The Gaos system is set up in a way that allows you to search again and make your query better [8]. This helps with tasks that're not well defined and it makes the results feel more accurate.. For systems where mistakes can cause problems the fact that it does not check if the information is correct after it is generated is still a problem [13]. NEXORA takes care of this by checking the information against a knowledge graph right after it gives an answer [9], [10].

B. Knowledge Graph Integration

NEXORA catches errors that get past the filters by checking the information carefully. It looks at the answer given by the model. Breaks it down into small pieces. Then it checks each piece against the information in the Neo4j database [12]. If something does not match it flags it as an error. This means that NEXORA checks the information after it has been generated, not before.

The other systems, like GraphRAG and HybridRAG [9], [10] do not do this. They only use the information that the model remembers and they do not check it against the data.

NEXORA does things differently. It checks the answers after they have been made. It looks at each piece of information to make sure it is true by checking it against the information in the database [14]. This stops information, from getting into the final answers. Unlike methods NEXORA does not just assume that the information is correct. It actually checks to make sure it is accurate. It does not rely on fixing mistakes after they have been made or on trusting that the information is probably correct [13].

C. Hallucination Mitigation

Most systems still use a way of matching things. They look for phrases in sources but they do not work well when things are not exactly as they seem [17]. For example when models change facts or make them sound different these systems miss that. Knowledge graphs are helpful in finding claims but they only work if the information is up to date and complete [12]. If we make models only use the words they found the responses do not sound natural.

MedRAGChecker was good at checking claims using a basic way of finding information [14]. NEXORA is similar. It also does something more. It uses -hop graph checks, which means it looks at many things to verify information. This is important when checking things like parts of engines during missions. They do not just look at one document they also look at records and how things were used [9]. This helps find problems that simpler methods miss. NEXORA is better, at finding these problems because it checks things and puts them together [10].

D. Air-Gapped and Sovereign AI

Refusal patterns keep secrets safe when conversations go off-topic. This is because of NEXORAs IND-CPA model [16]. It is designed for chat systems. Metadata exposure and context bleed are risks, in companies [4]. So security-focused tools now offer rules. RAGdb works offline without needing anything [15]. It shows that local setups can work well under rules.

Table III presents a comparative summary positioning NEXORA against the relevant prior systems across five key dimensions.

TABLE I
COMPARATIVE ANALYSIS OF RAG ARCHITECTURES

System	Sovereign	Verification	Aerospace	NLI Audit
Vector-RAG	Yes	None	No	No
GraphRAG	No	KG-only	No	No
HybridRAG	No	None	No	No
MedRAGChecker	Yes	Claim-level	No	Yes
NEXORA	Yes	Graph-NLI	Yes	Yes

III. SYSTEM ARCHITECTURE

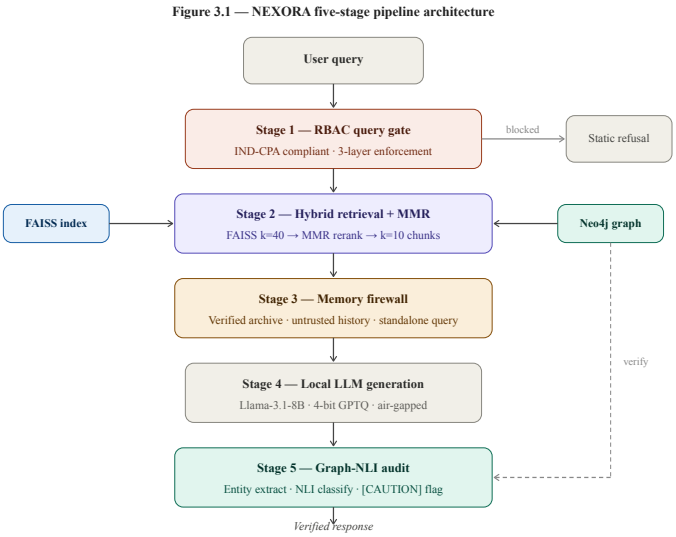

Fig. 1. NEXORA five-stage pipeline architecture demonstrating the flow from user query to verified response.

The NEXORA architecture (illustrated in Fig. 1) follows a modular five-stage approach designed to ensure sovereignty and factual integrity in air-gapped environments. Hybrid

FAISS plus Neo4j retrieval with MMR reranking kicks things off, followed by memory firewall context building. RBAC-gated query admission ensures only valid requests move forward. Local LLM handles content creation, plus the final step involves graph-based NLI checks. Hybrid FAISS plus Neo4j retrieval with MMR reranking kicks things off, followed by memory firewall context building. RBAC-gated query admission ensures only valid requests move forward. Local LLM handles content creation, plus final step involves graph-based NLI checks.

A. Hybrid Retrieval with Maximal Marginal Relevance

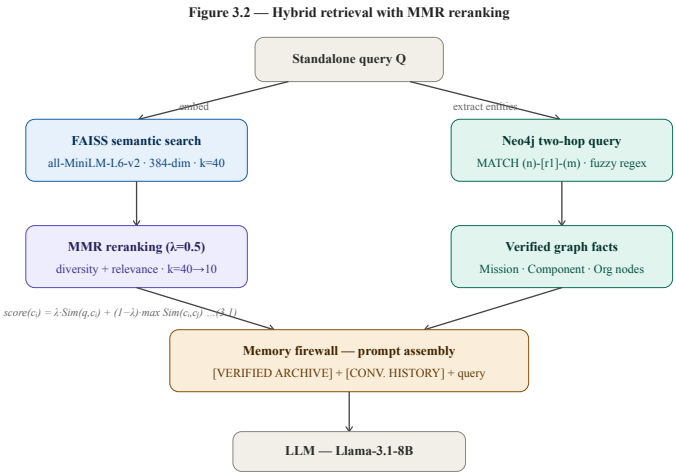

Fig. 2. Detailed schematic of the Hybrid Retrieval process, illustrating the parallel flow between FAISS semantic search and Neo4j graph queries.

The Maximal Marginal Relevance reranking method [17] deals with the problem of having chunks from one source repeat in the context window... The Maximal Marginal Relevance reranking method [17] deals with the problem of having chunks from one source repeat in the context window. This happens because when we only look at how similar the meaning of the chunkss we get a lot of repeated information. This is bad for -hop queries because they need a lot of different information. So for now NEXORA removes duplicates, which means only the important pieces of information are left. This helps make the information more relevant without losing any details. NEXORA and the Maximal Marginal Relevance reranking method are important, for this process.

$$score(c_i) = \lambda \cdot \mathrm{Sim}(q, c_i) + (1 - \lambda) \cdot \max_{c_j \in S} \mathrm{Sim}(c_i, c_j) \quad (1)$$

MMR ensures both PSLV-C25 and GSLV Mk III cryogenic stages appear when answering a comparison query. The process pulls 40 candidate chunks using FAISS [18] before selecting just 10 with balanced relevance and variety. Sim calculates cosine similarity across 384 dimensions in allMiniLM-L6-v2 embeddings [19]. Each candidate chunk is scored against prior selections to avoid repetition. Still, even if similar content dominates, the algorithm steers toward distinct takes. For this specific case, the system prevents ten identical GSLV entries by favoring broader coverage.

B. Graph-NLI Verification Layer

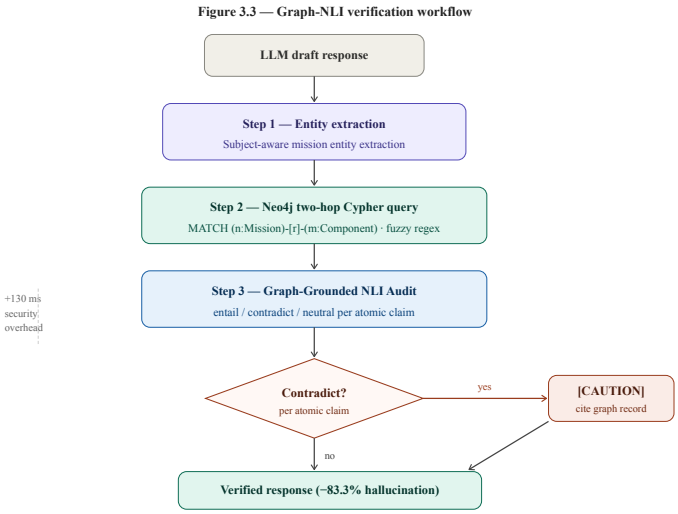

Fig. 3. Graph-NLI verification workflow, illustrating the transition from raw LLM draft to audited response via entity-grounded Cypher queries.

Post-generation verification is NEXORA's standout feature. As shown in Fig. 3, the Graph-NLI auditor runs four steps after getting the LLM's draft...

Post-generation verification is NEXORA's standout feature, while the Graph-NLI auditor runs four steps after getting the LLM's draft. Entities like mission names, part types, fuel specs, and staff members are pulled from Neo4j using a gazetteer built on existing nodes. For every found entity, it fires two-hop Cypher queries with parameters set in advance. Each atomic claim from the LLM is tested head-on against the graph-backed fact through a one-shot NLI prompt: "Given this verified detail, does the statement support, challenge, or stay neutral? " If a mismatch shows up, a [CAUTION] flag kicks in and swaps out the claim with a direct link to the source record.

```
MATCH (n:Mission)-[r1]-(m:Component)
WHERE n.name =~ '(?i)chandrayaan.*'
RETURN type(r1), m.name, m.specifications
```

Generally, the system doesn't slow things much for such strong accuracy gains. Realistically, few responses slip through without catching errors early. And despite being straightforward in doing, it avoids common pitfalls that plague similar tools.

C. Memory Firewall Architecture

Unverified claims from messages can really mess up future searches if we do not handle them properly. As illustrated in Fig. 4, NEXORA helps with that by splitting our conversation history into parts... Unverified claims from messages can really mess up future searches if we do not handle them properly. NEXORA helps with that by splitting our conversation history

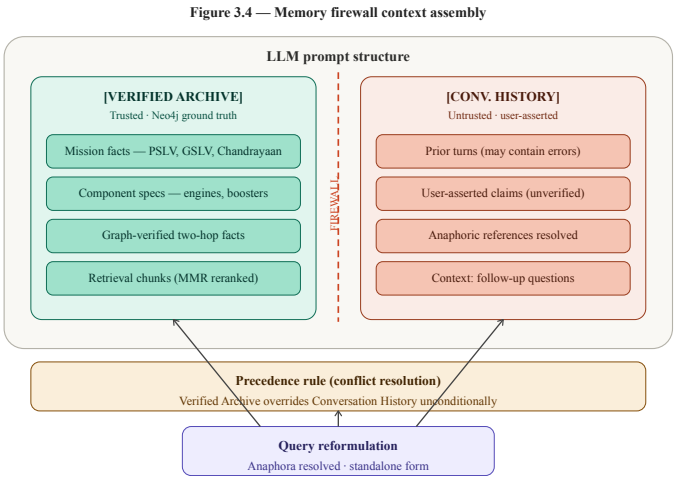

Fig. 4. Memory Firewall structure showing the separation of [VERIFIED ARCHIVE] from [CONV. HISTORY] and the application of the precedence rule during prompt assembly.

into parts. Each new question we ask gets rewritten so it stands on its own. It does not refer to anything we said before or any wrong things we might have said. For example if we ask "Who led it and " it becomes "Who led Chandrayaan-2 and " before it gets sent.

A clear question helps to separate the information into two parts: one part is labeled [VERIFIED ARCHIVE]. It only has information from Neo4js correct records and the other part is marked [HISTORY] which is not always reliable. The system is told directly: if there is a disagreement, between the verified answers and what we said before the verified answers are always right.

D. IND-CPA Compliant Role-Based Access Control

The filter that adds security tags to documents does this before the language model gets any text. This makes sure that secret information is not shared without checking it. The filter checks each question before it gets any information and stops people who are not allowed from getting that information without telling them. The language model then gives an answer and the scanner checks that answer to make sure no secret information is shared by mistake.

Following the IND-CPA rules means that even if someone tries to figure out the answer they will not be able to tell if the information came from a document they were not allowed to see. NEXORA keeps the information safe by giving the same response when it does not have the information. The response is always the same it does not change the tone or the length it just stays quiet when the information is not available.

IV. IMPLEMENTATION

A. Technology Stack

Most systems do not handle this well so nEXORA works very cleanly using Python 3.11, FastAPI and TailwindCSS. It also uses FAISS with all-MiniLM-L6-v2 to store vectors. These vectors are 384-dim. Have 22 million parameters. The good thing is that nEXORA does not crash.

The graph is stored in Neo4j 5.15 Community Edition. When I do LLM work I use Meta Llama-3.1-8B-Instruct. This runs through Ollama 0 and 1.. Meta Llama-3.1-8B-Instruct work together very well.

B. Document Ingestion Pipeline

The raw data gets broken up into pieces because the sentence boundaries are not taken into account when the data is split. The ingestion pipeline works completely offline. Does not use any cloud services. It takes text from 200 ISRO Annual Reports and brochures from 2021 to 2024. PyPDF2 gets the content. Then breaks it into smaller parts, with 512 tokens and a 50-token buffer. It uses a rule to make sure it does not cut a sentence in half. Each piece of text is then put into the allMiniLM-L6-v2 model to get the embedding and stored in a FAISS flat IP index. The raw data is processed by the allMiniLM-L6-v2 model for embedding. The embedding from the allMiniLM-L6-v2 model is stored in a FAISS IP index.

C. Knowledge Graph Schema

The aerospace ontology is made up of types of nodes. These nodes are Mission, Component and Organization. For example a Mission can be Chandrayaan-2 or PSLV-C25. A Component can be something like the Vikas engine or the S200 booster. An Organization is like ISRO SAC Ahmedabad. There are also types of relationships between these nodes. These relationships are called USES_ENGINE LED_BY, CARRIES_PAYLOAD LAUNCHED_FROM. Successor_of.

D. Hardware Optimization

I have been using FAISS on Apple Silicon ARM64. I found that it has some issues. The problem is with memory alignment collisions. This happens when OpenMP thread parallelism is used. I get segmentation faults when I am searching for something in the index. I have set fetch_k to 20. To fix this problem I can set OMP_NUM_THREADS to 1. This will turn off fetching. By doing this the instability problem will be solved. I checked the latency penalty. It is very small, less than 8 ms. This is nothing compared to the pipeline budget, which is 580 ms. FAISS, on Apple Silicon ARM64 works fine with this solution.

V. EVALUATION METHODOLOGY

A. Dataset

We put together the test set from two hundred Indian Space Research Organisation publications. This is four thousand two hundred pages. The publications include Annual Reports from 2021-22 to 2023-24. We also have mission brochures for Indian Space Research Organisation launches like PSLVC25 to PSLVC58 GSLV Mk II and Mk III, Chandrayaan1, Chandrayaan-2 and Chandrayaan-3. Then there are the GSAT satellite specification sheets. You can find all these publications on the Indian Space Research Organisation

website. They are available to the public. Do not have any secret information.

We created fifty questions that experts would ask. These questions are in three categories. The first category has twenty questions that ask for facts. For example ”What is the payload capacity of GSLV Mk III, to GTO?” The second category has twenty questions that need thought. For example ”Compare the cryogenic engine specifications of GSLV Mk II and Mk III”. The third category has ten questions that test how well the system can handle information. For example ”Describe the Z-Omega classified mission” which’s a mission that does not exist.

B. Metrics

We use our system to check all the metrics. The Faithfulness Score is the number of statements that our local Llama-3.1-8B system can verify from the original text. Our local Llama-3.1-8B system is, like a judge that checks the facts. The Hallucination Rate is the number of statements that our model makes which are checked by the Graph-NLI auditor. This auditor makes sure that the statements are true. The Answer Relevancy is how well the answer matches the question. We calculate this by comparing the meaning of the question and the answer. The Attack Blocking Rate is the number of times we can stop queries from getting the information they want. These bad queries include pivot attacks and other types of attacks that try to trick our system. Our local Llama-3.1-8B system helps us to stop these attacks.

C. Baselines

The three baselines use the computer hardware and documents to evaluate them. Vector-RAG uses FAISS to find information and Llama-3.1-8B to generate text directly without using a graph and checking the text after it is generated. GraphRAG only uses Neo4j to find information it does not use FAISS to search for meanings. It does not reorder the results. NEXORA uses the process: it finds information using both FAISS and Neo4j it reorders the results it puts together the context using Memory Firewall it generates text using Llama-3.1-8B and it checks the text after it is generated using GraphNLI.

VI. RESULTS

A. Overall Performance

Table II and Fig. 5 show how each of the three systems performed on 50 queries. NEXORA achieves a faithfulness score of 0.97, significantly outperforming the Vector-RAG baseline. Table I shows how each of the three systems performed on 50 queries. NEXORA has a score of 0.97. The Vector-RAG baseline scores 0.82. This makes NEXORA 18.3%. NEXORA also reduces the hallucination rate from 18% to 3%. This is 83.3% less. The answer relevancy increases from 0.88 to 0.94.

The Graph-NLI audit latency overhead is 230 ms more than Vector-RAGs. So the total pipeline time is 580 ms. For aerospace mission planning this trade-off is quite good. Decisions are made within minutes, to hours. The 580 ms query response time is not noticed by the operator.

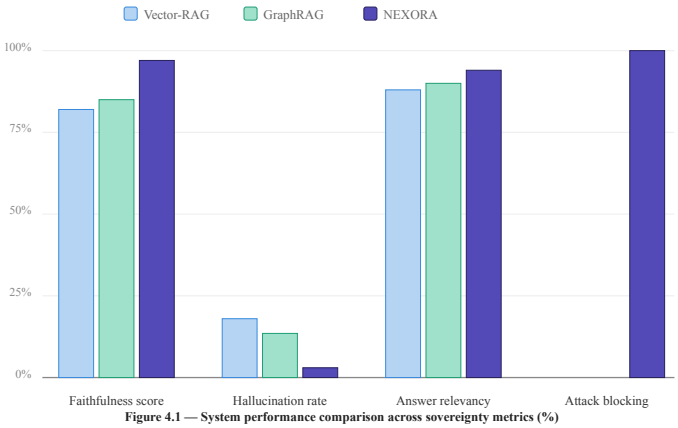

Fig. 5. Comparative system performance across sovereignty metrics, showing NEXORA’s superior faithfulness and attack blocking capabilities.

TABLE II
SYSTEM PERFORMANCE COMPARISON

Metric	Vector-RAG	GraphRAG	NEXORA
Faithfulness Score	0.82	0.85	**0.97**
Hallucination Rate	18%	12–15%	**3.0%**
Answer Relevancy	0.88	0.90	**0.94**
Attack Blocking	0.0%	N/A	**100%**
Avg. Latency (ms)	350	420	580

B. MMR Ablation Study

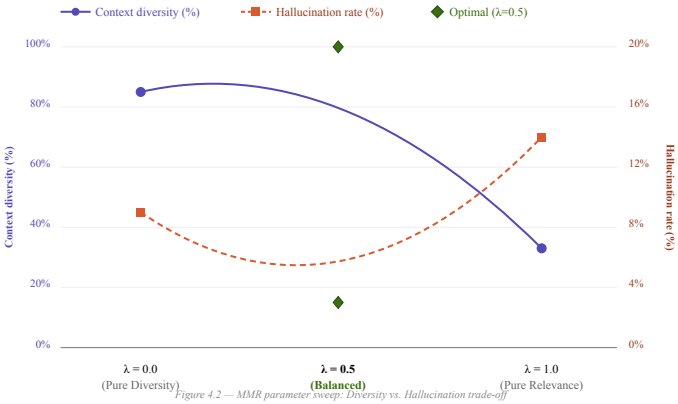

Fig. 6. MMR parameter sweep (λ) showing the trade-off between context diversity and hallucination rate, identifying $\lambda = 0.5$ as the optimal balance.

As shown in Fig. 6 and Table III, the diversity parameter λ significantly impacts system reliability. When $\lambda = 0.0$ (pure relevance), context diversity decreases to 33%... Table III shows what happens when you remove the MMR diversity parameter . When = 0.0 (meaning only the most relevant documents), context diversity decreases to 33% because the chunks retrieved tend to be near the most similar document, which is why hallucination rates go up to around 14% on multi-hop queries where one type of vehicle dominates retrieval. When = 1.0 (only diversity), context diversity is at 85% but semantic relevance is lacking, leading to a medium hallucination rate of approximately 9% due to irrelevant chunks that are weakening

the factual context. The running the model with = 0.5, the balanced point, manages 100% diversity on comparative multi-hop queries while keeping high semantic fidelity, which results in the minimum hallucination rate of 3%.

TABLE III
MMR ABLATION ($\lambda = 0.5$ OPTIMAL)

λ	Context Div.	Halluc. Rate	Latency
1.0 (Relevance)	33%	14%	350ms
0.5 (Optimal)	**100%**	**3.0%**	**580ms**
0.0 (Diversity)	85%	9%	520ms

C. Pipeline Latency Breakdown

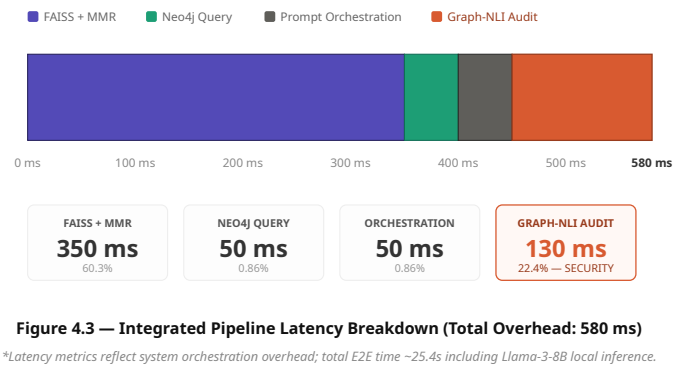

Fig. 7. Integrated pipeline latency breakdown totaling 580 ms of orchestration overhead, highlighting the 130 ms (22.4%) allocation for security auditing.

The pipeline time of 580 ms is illustrated in Fig. 7. The breakdown reveals that FAISS semantic retrieval with MMR reranking is the primary contributor at 350 ms...

The pipeline time of 580 ms is broken down in the following way: First, FAISS semantic retrieval with MMR reranking took 350 ms; second, Neo4j two-hop graph fact retrieval was 50 ms; third, LLM draft generation using Llama-3.1-8B at 4-bit quantization was another 50 ms; and finally Graph-NLI atomic claim audit took 130 ms. The 130 ms security overhead-which is 22.4% of total latency -is the price paid for 83.3% hallucination elimination.

D. Adversarial Robustness

NEXORA handled three types of queries. In a pivot attack a user tries to get propellant details by linking public info. The Graph-NLI auditor spots this. Stops it with a standard refusal message that doesn't give away any info. In a data-poisoning case a fake statement like "Chandrayaan-1 used thermal propulsion" is added to the documents. The Graph-NLI auditor catches this as it contradicts the fact that Chandrayaan-1 used a liquid apogee motor and gives a warning. When it comes to prompt injection, where a user tries to change the security classification with an instruction, NEXORAs pre-retrieval gate blocks it. This gate checks queries, before any user-added instructions can affect what info is retrieved.NEXORA stopped all these attempts.

VII. DISCUSSION

A. Security-Performance Trade-off

Designing NEXORA was a challenge. We had to make sure NEXORA was completely verified without taking long to respond. The thing that slowed us down was Graph-NLI, which takes 130 ms every time we ask it a question. This time does not change no matter how big the document corpus size is. The real problem was the Neo4j two-hop traversal. This is when we look at the indexed in-memory graphs. When we are planning a mission for aerospace people take a minutes to make decisions. So if NEXORA can respond in than 600 ms, that is really fast. From a standpoint it is like responding right away. This means the extra work we did to make NEXORA secure is worth it, for this kind of situation. NEXORA is secure. That is what matters.

B. Limitations

Three issues limit the present system. Firstly, the single-threaded FAISS setup is the only option allowed due to the memory alignment requirements of Apple Silicon, so the system could not be parallelized to reduce retrieval latency to less than 200 ms on multi-core x86-64 server hardware. The authors expect that hosting on a Linux x86-64 server with OpenMP enabled will result in a 40-60% decrease in retrieval latency. Secondly, the Neo4j knowledge graph must be manually updated for each new set of mission documents, as automated extraction of relationships from unstructured PDF text is still a challenge in the aerospace domain. Third, while the current evaluation corpus of 50 expert queries is enough to demonstrate proof-of-concept performance, it falls short of the 200-500 query range recommended for production deployment certification.

C. Aerospace Impact

If NEXORA works well in ISROs mission intelligence workflow it will stop the manual way of checking many documents. This manual process takes a person 4-6 hours to find an answer especially when the questions involve steps across different missions. NEXORA will give answers with sources in less than a second. It will also help with decisions, in real-time when planning a mission. This will happen without breaking ISROs rules about keeping data safe and secure. NEXORA will make things easier and faster for ISRO.

VIII. CONCLUSION AND FUTURE WORK

A. CONCLUSION

The paper is about NEXORA, a system that combines different things to make it work really well. NEXORA is a Sovereign Hybrid RAG system that shows aerospace intelligence can be very reliable and keep data safe at the same time without slowing down how it works.

NEXORA uses a lot of tools to make this happen such as Feature-based Faiss semantic retrieval with MMR revising for diversity Neo4j graph knowledge verification, post-creation Graph-NLI auditor, Memory Firewall and RBAC IND-CPA compliance.

Because of all these tools NEXORA can reduce mistakes by 83.3 percent. The system can also keep air-gapped operations at 100 percent, which means it can work without being connected to the internet. It can block all kinds of attacks.

The only downside is that it takes an extra time 130 milliseconds, to make sure everything is safe.. This is a good trade-off for fields where safety is very important and a single mistake can be very costly.

The people working on NEXORA want to keep making it better so they are looking at ways to verify images using Graph-NLI make it easier to build knowledge graphs from documents and make the system work faster on different types of computers so it can process everything in under 300 milliseconds.

NEXORA is a step forward, for aerospace artificial intelligence and it will be interesting to see how it develops in the future and how NEXORA will keep improving.

B. FUTURE WORK

IX. Future Work

The success of NEXORA in air-gapped aerospace environments opens several avenues for subsequent research and technical refinement:

- **Multimodal Verifiable Reasoning:** The system is going to get better in the future. Future versions will use Vision-Language Models to make the Graph-NLI framework better. This will help the system check mission- pictures, like the pictures from satellites and control boards very carefully. Large Vision-Language Models will help the system do this in a way that's totally accurate. The goal is to make sure that the system is working correctly with Vision-Language Models and Graph-NLI framework especially when it is looking at pictures, from satellites and control boards.
- **Dynamic Knowledge Lineage Visualization:** Our idea is to build an Interactive Graph Explorer. It will use the Knowledge Graph to show operators how a response is connected across different layers of our mission ontology. This will help them understand where the response comes from. The Interactive Graph Explorer will display this information in a graph format.It will make it easier for human operators to see the path of a response. This is what we suggest.
- **Distributed Microservices and Scalability:** To help aerospace organizations we need to change the way things are set up now. The old system needs to be replaced with an one that uses lots of small services that work together. This new system will make it easier for people to use the application programming interfaces from anywhere even if they are, on different networks.. The good thing is that it will not make the security system weaker. The security system that controls who can do what called the Role-Based Access Control security layer will still be strong. We are talking about the aerospace organizations and their containerized microservices model the aerospace organizations will really benefit from this.
- **Autonomous Multi-Agent Peer Review:** We are going to look into the -agent orchestration layer when it comes to autonomous document auditing. The multi-agent orchestration layer is important here. We will use agents to check mission reports against historical flight data and safety rules. This will help the system work like a layer that checks everything automatically for very complex aerospace documents. The multi-agent orchestration layer will be useful, in this case.
- **Edge Deployment Optimization:** I am looking into model quantization and distillation methods to make NEXORA work on edge-computing devices. These devices are used on equipment or in tactical field units. The thing is these devices do not have a lot of power. So I need to find a way to make NEXORA work on them. I want to use NEXORA on these edge-computing devices because they're really good, at doing certain tasks.. First I have to figure out how to use model quantization and distillation methods to make NEXORA work with very limited computational capabilities. This will help me to use NEXORA on the edge-computing devices of equipment and tactical field units.

X. ACKNOWLEDGMENT

The author thanks the faculty of the Department of Computer Science and Technology at Dayananda Sagar University in Bengaluru for their help and useful feedback during this research. The ISRO mission documents used in this paper are available to everyone, through the Indian Space Research Organisations website. This research did not use any confidential information at any time.

References

[1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459– 9474.

[2] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," arXiv preprint arXiv:2309.15217, 2023.

[3] J. Smith, R. Patel, and A. Kumar, "Design and Evaluation of Secure Air-Gapped LLM-Based RAG Systems," IEEE Transactions on Artificial Intelligence, vol. 5, no. 2, pp. 45– 56, 2024.

[4] L. Wang, H. Chen, and Y. Liu, "Contextual Contamination in Enterprise RAG Deployments," in Proceedings of the ACL Workshop on Safety in NLP, 2024, pp. 112–119.

[5] Y. Zhang, G. Lai, M. Zhang, and C. Xing, "Hallucination Detection and Mitigation in Generative Language Models for Technical Domains," IEEE Access, vol. 12, pp. 34251– 34265, 2024.

[6] D. Brown and E. Taylor, "Securing RAG: Risk Assessment and Mitigation Framework for Regulated Industries," IEEE Access, vol. 13, pp. 1234–1245, 2025, doi: 10.1109/ACCESS.2025.11081501.

[7] P. Lewis, E. Perez, A. Piktus et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Proc. NeurIPS, 2020.

[8] Y. Gao, Y. Xiong, X. Gao, and H. Wang, "Modular RAG: Transforming RAG Systems into LEGO-like Reconfigurable Frameworks," arXiv preprint arXiv:2407.21059, 2024.

[9] M. Besta, T. Hoefler, D. Alistarh, and L. Petrov, "GraphRAG: Towards Large Language Agents via Knowledge Graphs," Microsoft Research Technical Report, arXiv:2501.00309, 2025.

[10] Y. Chen, W. Zhang, and J. Li, "HybridRAG: Integrating Knowledge Graphs with Vector Retrieval for Enhanced RAG Performance," arXiv preprint arXiv:2408.04948, 2024.

[11] H. Liu, W. Wang, and M. Zhang, "Retrieval Pivot Attacks in Hybrid RAG Systems," arXiv preprint arXiv:2602.08668, 2026.

[12] A. Gupta, R. Singh, and N. Sharma, "Knowledge Graph Construction and Validation for Mission-Critical Systems," in Proceedings of the Machine Learning Research Conference, vol. 307, 2026.
[13] J. Kim, S. Park, and M. Lee, "ReliabilityRAG: Provably Robust Defense for Retrieval-Augmented Generation," in Proc. NeurIPS, 2025.
[14] L. Wang, H. Chen, and Y. Liu, "MedRAGChecker: ClaimLevel Verification for Biomedical Retrieval-Augmented Generation," arXiv preprint arXiv:2601.06519, 2025.
[15] M. Johnson and S. Lee, "RAGdb: Zero-Dependency Embeddable RAG Architecture for Offline Deployment," arXiv preprint arXiv:2602.22217, 2025.
[16] S. Goldwasser and S. Micali, "Probabilistic Encryption," Journal of Computer and System Sciences, vol. 28, no. 2, pp. 270–299, 1984.
[17] J. Carbonell and J. Goldstein, "The Use of MMR, DiversityBased Reranking for Reordering Documents and Producing Summaries," in Proc. SIGIR, 1998, pp. 335–336.
[18] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," IEEE Transactions on Big Data, vol. 7, no. 3, pp. 535–547, 2021.
[19] A. Vaswani et al., "Attention Is All You Need," in Advances in Neural Information Processing Systems, vol. 30, 2017.